

Requirements: Functional & Non-Functional

These requirements govern the AI / agentic (LangGraph) system for storage capacity management and deviation detection across a heterogeneous multi-OS fleet of at least 500 servers (AIX, all Linux distributions, and Windows Server). The fleet is treated as a single, unified problem: every operating system is first-class and delivered together in a single release.

Functional Requirements (FR) describe *what* the system must do. Non-Functional Requirements (NFR) describe *how well* it must do it (quality attributes and measurable targets). All capacities are exact bytes; deterministic rules perform all measurement and detection; the LLM layer only explains, prioritizes, and recommends; and the system is advisory and recommend-only, automatically flagging low-confidence or high-impact findings and withholding them from auto-confirmation rather than executing remediation or pausing for any human step.

Priority legend: Must (mandatory for the release) · Should (high value, expected) · Could (desirable if capacity allows). **Linked Objective** refers to the five project objectives: **01** deviation detection vs standards · **02** optimization (useless / obsolete / misplaced data) · **03** reliability & full-filesystem prevention · **04** operational efficiency & forecasting · **05** risk reduction & compliance.

Functional Requirements

Id	Requirement	Priority	Linked Objective	Acceptance Criteria
FR-01	The system MUST ingest one collection-run envelope file per run (an object { <code>snapshot_id</code> , <code>generated_at</code> , <code>host_count</code> , <code>hosts[]</code> }) and validate both the envelope and each <code>hosts[]</code> element against the single shared, OS-agnostic storage collection schema, regardless of each host's <code>platform</code> (<code>aix</code> <code>linux</code> <code>windows</code>).	Must	01, 04	Given an envelope for any mix of OS, ingestion validates the run-level fields (<code>snapshot_id</code> , <code>generated_at</code> , <code>host_count</code>) and confirms <code>host_count</code> equals the number of elements in <code>hosts[]</code> ; for each host it validates the required identity fields (<code>host_id</code> , <code>hostname</code> , <code>platform</code> , <code>os_family</code> , <code>os_distribution</code> , <code>os_version</code> , <code>scan_timestamp</code>). A malformed envelope, a mismatched <code>host_count</code> , or any schema-violating host element is rejected with a typed error and never partially persisted. JSONL input (one host object per line, each line carrying <code>snapshot_id</code> + <code>scan_timestamp</code>) is accepted and validated against the same schema.
FR-02	The system MUST collect storage state by branching the collection command per <code>platform</code> , with AIX, Linux, and Windows implemented at the same depth and obligation level in a single release.	Must	01, 03, 04	AIX uses <code>lsvg / lslv / lspv / df -k / df -v / lspv -a</code> ; Linux uses <code>df / df -i / vgs / lvs / pvs --units b / tune2fs / xfs_info / btrfs</code> ; Windows uses WinRM + PowerShell (<code>Get-Volume</code> , <code>Get-Partition</code> , <code>Get-Disk</code> , <code>Get-PhysicalDisk</code> , <code>Get-StoragePool</code> , <code>Get-VirtualDisk</code> , <code>Win32_PageFileUsage</code> , <code>Get-Acl</code>). Each branch emits host documents conforming to the single shared schema; a missing Windows code path fails CI exactly as a missing AIX or Linux path does.
FR-03	The Windows collector MUST connect over WinRM and execute PowerShell to populate <code>filesystems[]</code> , <code>storage_topology</code> , and <code>paging[]</code> as a first-class platform alongside AIX and Linux.	Must	01, 03, 04, 05	For a Windows host, NTFS/ReFS volumes (drive letters and folder mount points) appear in <code>filesystems[]</code> with sizes in exact bytes; Storage Spaces pools map to <code>storage_topology.groups[]</code> (<code>group_kind</code> = <code>storage-pool</code>) and virtual disks to <code>storage_topology.volumes[]</code> (<code>volume_kind</code> = <code>virtual-disk</code>); basic disks yield <code>partition</code> volumes with empty topology groups; the pagefile appears in <code>paging[]</code> .
FR-04	The system MUST normalize all capacity values to exact bytes on every platform and field, performing no lossy unit rounding.	Must	01, 03, 04	All size, reserved, extent-size, and paging size fields are integers in bytes; AIX <code>df -k / df -v</code> kilobytes (and any KB/block-based PowerShell values) are converted exactly at collection time; round-trip checks confirm no precision loss. No value is ever stored as blocks, KB, MB, GB, or a percentage.
FR-05	The system MUST represent every field that is not applicable to a given OS or filesystem type as an explicit <code>null</code> (verified N/A) rather than omitting it, distinguishing "not applicable / unknown" from "not collected" or "malformed".	Must	01, 04	<code>reserved_bytes</code> is <code>null</code> on <code>xfs/jfs2/ntfs/refs/btrfs</code> and populated on ext filesystems; <code>inodes_total</code> / <code>inodes_used</code> are <code>null</code> on Windows and on <code>btrfs</code> (SUSE/SLES); <code>mount_options</code> is <code>null</code> on Windows; <code>acl_summary</code> is <code>null</code> on Unix; AIX-specific capability fields are <code>null</code> where not applicable. An explicit <code>null</code> on a nullable (COND) field passes validation; a field that is required (REQ) yet absent is rejected.
FR-06	The system MUST derive <code>storage_class</code> (system vs data) per filesystem downstream from the	Must	01, 05	AIX rootvg-backed and Linux root/system-volume filesystems classify as <code>system</code> ; AIX datavg and non-root Linux filesystems classify as <code>data</code> ; the Windows boot/install volume (typically

Id	Requirement	Priority	Linked Objective	Acceptance Criteria
	backing_ref / group_ref linkage, generalized across all OS including Windows. storage_class is NOT collected.			C:) classifies as system and other drives as data . Classification is reproducible from the host document alone, without re-collection.
FR-07	The system MUST compute btrfs (SUSE/SLES) true usage in a subvolume- and snapshot-aware manner via qgroups, not from plain df .	Must	01, 03	For btrfs filesystems, used/available bytes are derived from btrfs filesystem usage / qgroups rather than df ; the document records the source so a SUSE host with subvolumes reports true bytes; a deliberate df -vs-qgroup discrepancy test selects the qgroup value.
FR-08	The system MUST link filesystems to storage topology via backing_ref / group_ref keys (weak coupling by key, not nesting) and expose this linkage so downstream derivation can resolve it.	Must	01, 04	Each filesystem backed by an lvm-lv , virtual-disk , or storage-pool-backed volume resolves its backing_ref / group_ref to an entry in storage_topology ; unresolved references are reported as data-quality defects; plain-partition Linux and basic-disk Windows correctly carry empty topology groups.
FR-09	The system MUST perform layered reference matching of each host / filesystem / group against per-OS reference standards in priority order, emitting an explicit no_reference when no layer matches.	Must	01, 04, 05	Matching attempts layers from most specific (e.g. host_role_env + os_distribution + mount point) to least specific (platform default); the selected layer and its precedence are recorded; when no layer applies, the item is marked no_reference and excluded from deviation scoring rather than silently defaulted. (The reference standards / golden images themselves are an upcoming deliverable.)
FR-10	The system MUST detect deviations versus standards for both filesystems and storage groups.	Must	01, 03, 05	For each matched item, the engine flags size / threshold / fstype / mount-option / redundancy deviations with a severity and the reference layer used; AIX, Linux, and Windows items are all evaluable; results are reproducible for identical inputs. Deviation severities are derived downstream, not collected.
FR-11	The system MUST detect storage optimization opportunities (useless, obsolete, or misplaced data) using only the non-sensitive storage metadata in the schema.	Should	02, 05	The engine flags candidates such as large reclaimable capacity, likely-obsolete data, over-provisioned volumes, and application data residing on system -class storage that belongs on data , each with a rationale, and without accessing filenames, owners, or file contents.
FR-12	The system MUST flag inappropriate data on system storage for risk/compliance, including the Windows system volume (typically C:).	Must	05	Filesystems classified system that carry data-class content are flagged as risk findings, each citing the originating standard; Windows system-volume violations are surfaced identically to AIX rootvg and Linux root.
FR-13	The system MUST produce capacity forecasts and growth trends per filesystem and group across snapshots, in bytes on every platform, and in inodes where inode accounting exists (Unix filesystems with a fixed inode table).	Should	03, 04	Given at least two snapshots per host from the append-only UTC time series, the system estimates time-to-full / projected utilization in bytes on all platforms, and additionally in inodes where inodes exist, with a confidence indicator; filesystems near saturation are prioritized; byte forecasts are emitted for every platform.

Id	Requirement	Priority	Linked Objective	Acceptance Criteria
FR-14	The system MUST capture a non-sensitive Windows ACL summary (<code>acl_summary</code>) for access compliance, never persisting raw or sensitive ACLs.	Must	O5	Windows volumes carry <code>acl_summary</code> capturing non-sensitive access posture; compliance checks evaluate it against standards; no raw ACE / SID-level data, named ACEs, owners, or filenames are stored or emitted.
FR-15	The system SHOULD use an LLM / agentic (LangGraph) layer to generate explanations, prioritizations, and remediation suggestions grounded in deterministic findings.	Should	O1, O2, O3, O4, O5	LLM outputs cite the specific finding, reference layer, and metrics they rely on; no recommendation is produced without an underlying deterministic finding; outputs are advisory only and never alter measured values.
FR-16	The system MUST apply automated confidence-based triage so that material or low-confidence findings are flagged and withheld from auto-confirmation, with no remediation executed and no human approval step in the pipeline.	Must	O1, O2, O5	Findings above a severity/impact threshold or below a confidence threshold are automatically flagged, withheld from auto-confirmation, and surfaced distinctly from auto-confirmed findings; the triage decision is recorded with the governing threshold and timestamp.
FR-17	The system MUST generate fleet-level and host-level reports covering deviations, optimization, forecasts, and risk, segmented by OS.	Must	O1, O2, O3, O4, O5	Reports aggregate findings across at least 500 hosts, are filterable by <code>platform</code> / <code>os_distribution</code> / <code>host_role_env</code> / <code>storage_class</code> , and each finding traces to its host, its collection run (<code>snapshot_id</code>), and its reference layer.
FR-18	The system SHOULD capture feedback on findings and recommendations to improve future matching and detection.	Should	O1, O4	Users can confirm, dismiss, or correct findings; feedback is stored with the finding id and reference layer and is retrievable to tune standards and suppress recurring false positives.
FR-19	The system MUST record <code>collection_method</code> , <code>scan_confidence</code> , and <code>scan_timestamp</code> per host, and surface low-confidence or stale snapshots.	Must	O3, O4	Each host document carries this provenance; collection runs below a confidence threshold or older than a freshness window are flagged in detection and reporting and may be excluded from forecasting.
FR-20	The system MUST normalize <code>os_family</code> / <code>os_distribution</code> so RHEL, Oracle Linux, Rocky, and AlmaLinux share XFS/LVM standards, while Debian/Ubuntu (ext4) and SUSE/SLES (btrfs) apply their own.	Must	O1, O4	A host tagged Oracle Linux is matched against the same RHEL-family XFS standard set (no <code>reserved_bytes</code> expected); Debian/Ubuntu match ext4 standards (with <code>reserved_bytes</code>); SUSE/SLES match btrfs subvolume-aware standards.

Non-Functional Requirements

Id	Category	Requirement	Target/Metric
NFR-01	Portability	A single schema and pipeline MUST support AIX, RHEL, Oracle Linux, Rocky, AlmaLinux, Debian, Ubuntu, SUSE/SLES (btrfs), and Windows as first-class platforms delivered together.	100% of the five objectives evaluable on every listed OS; all listed operating systems delivered in a single release, with Windows collection and standards shipped together with AIX and Linux.
NFR-02	Portability	Per-platform collection branches MUST be independently testable with representative fixtures for each OS / filesystem type.	Golden-fixture coverage for AIX/JFS2, XFS, ext4, btrfs, NTFS, and ReFS; CI fails if any platform branch lacks a passing fixture.
NFR-03	Performance	Ingestion, validation, and deviation detection of a single host document MUST complete within an interactive budget.	<= 2 s p95 per host document end-to-end (ingest + validate + match + detect), excluding LLM steps.
NFR-04	Performance	A full fleet detection pass MUST complete within an operational window.	>= 500 host documents processed in <= 15 min on the reference deployment; linear or better scaling.
NFR-05	Scalability	The system MUST scale to a fleet of at least 500 servers with mixed OS and multi-snapshot history.	Sustained ingestion of 500+ hosts, each with N appended snapshots, without architectural change; horizontal scale-out supported.
NFR-06	Security/Privacy	The system MUST process non-sensitive metadata only; no filenames, owners, or file contents are collected, stored, or transmitted.	Static and runtime checks confirm only non-sensitive capacities, topology, mount options, and aggregate metadata are persisted; presence of file-content, filename, or owner data fails the pipeline.
NFR-07	Security/Privacy	Windows collection MUST persist only the non-sensitive <code>ac_l_summary</code> , never raw ACLs / ACEs / SIDs.	Stored <code>ac_l_summary</code> contains no raw ACE, SID, or named-principal detail; an automated check rejects any document containing raw ACL data.
NFR-08	Security/Privacy	All remote collection MUST be encrypted, including WinRM for Windows.	WinRM uses an encrypted transport (HTTPS); SSH is used for Unix; plaintext transport is rejected; credentials are never stored in documents.
NFR-09	Reliability	Collection and ingestion MUST degrade gracefully on partial failure, never producing partially-persisted documents.	Partial collection yields a flagged-incomplete document or a controlled failure; persistence is atomic (all-or-nothing) per host within a collection run.
NFR-10	Maintainability	Standards, reference layers, and OS branches MUST be configurable and extensible without changes to core engine code.	A new OS standard or reference layer is added via configuration; adding a distribution that reuses a family standard requires no engine code change.

Id	Category	Requirement	Target/Metric
NFR-11	Observability	The pipeline MUST emit metrics and structured logs for collection, ingestion, matching, and detection per platform.	Per-stage success/failure counts, latencies, and <code>no_reference</code> rates exposed; per-platform dashboards available.
NFR-12	Auditability	Every finding and recommendation MUST be traceable to host, collection run, and reference layer.	100% of findings carry lineage (<code>host_id</code> , <code>snapshot_id</code> , reference layer, rule id); the audit trail is immutable.
NFR-13	Data Quality	Capacity data MUST be lossless (exact bytes), the filesystem-to-topology linkage MUST be verifiable, and full collection lineage MUST be present.	All sizes are integer bytes with no rounding; unresolved <code>backing_ref</code> / <code>group_ref</code> are flagged as defects; lineage (<code>collection_method</code> , <code>scan_confidence</code> , <code>scan_timestamp</code>) is present on every host document and the run identifiers (<code>snapshot_id</code> , <code>generated_at</code> , <code>host_count</code>) on every envelope.
NFR-14	Data Quality	btrfs sizing MUST be reconciled against qgroups so reported bytes are true, not <code>df</code> -misleading.	For btrfs hosts, reported used/available bytes match qgroup-derived values within tolerance; <code>df</code> -only sizing is rejected for btrfs.
NFR-15	Cost	The pipeline MUST bound LLM usage to control cost without degrading deterministic detection.	Deterministic detection runs with zero LLM cost; the LLM is invoked only for explanation/prioritization on filtered findings; a per-run token/cost budget is enforced.
NFR-16	Usability	Reports and review interfaces MUST let operators filter, drill down, and act on findings across the mixed fleet.	Operators filter by <code>platform</code> / <code>os_distribution</code> / <code>host_role_env</code> / <code>storage_class</code> and drill from fleet to host to filesystem to reference in <= 3 interactions.
NFR-17	Reliability	Snapshot freshness and confidence MUST gate downstream use.	Stale (beyond freshness window) or low- <code>scan_confidence</code> snapshots are flagged and optionally excluded from forecasting/scoring per policy.

Related upcoming deliverables

The following are explicit next steps and are out of scope for this requirements document; they are referenced here so dependent requirements are traceable: the reference standards / golden images (FR-09, FR-10, FR-12, FR-20), the OS-by-OS field-applicability matrix (FR-05), the field-by-field rationale plus Ansible collection (FR-02, FR-04), the LangGraph architecture (FR-15, FR-16), and a machine-validatable JSON Schema file (FR-01).